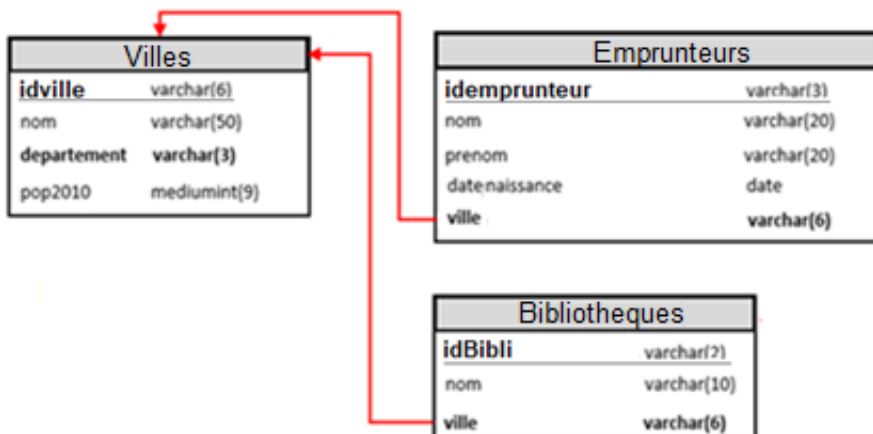


BD1(LMD) – TP5

Les jointures

EXERCICE 1 (sur la base **exemples**.)

On s'intéresse aux 3 tables ci-dessous situées dans la base **exemples** :



1) Ecrivez à l'aide de JOIN une requête permettant d'afficher le nom de la ville dans laquelle se trouve chaque bibliothèque.

Aperçu attendu :

nom de la bibliothèque	nom de la ville où la bibliothèque se trouve
VAILLAND	LE HAVRE
CAMUS	LE HAVRE

```

SELECT Bibliotheques.nom "nom de la bibliothèque",
       Villes.nom "nom de la ville où la bibliothèque se trouve"
FROM Villes JOIN Bibliotheques ON ville=idville;
  
```

2) Trouvez un énoncé dont la réponse serait cette requête :

```

SELECT *
FROM Bibliotheques NATURAL JOIN Villes
  
```

Affiche la bibliothèque et la ville qui portent le même nom, si elles existent !

3) Ecrivez à l'aide de JOIN une requête permettant d'afficher le nom de la ville dans laquelle se trouve chaque emprunteur.

aperçu attendu :

nom	prenom	nom
COURJAULT	JÉRÉMY	MARSEILLE
PLAZAT	JULIEN	MARSEILLE
BENICHO	ANTHONY	LYON
OLLIVIER	CARLOS	LYON
GUILLEMAIN	FABRICE	NICE

```
SELECT Emprunteurs.nom,prenom,Villes.nom
FROM Villes JOIN Emprunteurs ON ville=idville;
```

4) Trouvez un énoncé dont la réponse serait cette requête :

```
SELECT *
FROM Emprunteurs NATURAL JOIN Bibliotheques
```

Dans emprunteurs et bibliothèques, il y a 2 attributs qui portent le même nom : ville et nom

Donc si on faisait une jointure naturelle, on testerait l'égalité du nom de l'emprunteur avec le nom de la bibliothèque ainsi que la ville de l'emprunteur avec la ville de la bibliothèque !

Pour que cette double égalité soit vraie, il faudrait qu'il y ait un emprunteur qui s'appelle X qui habite à Bourg-en-Bresse et que X soit également une bibliothèque de Bourg-en-Bresse. C'est le cas pour Aimeric CAMUS !

5) En utilisant une jointure interne (JOIN), affichez pour chaque emprunteur, la liste des bibliothèques présentes dans la ville où il habite.

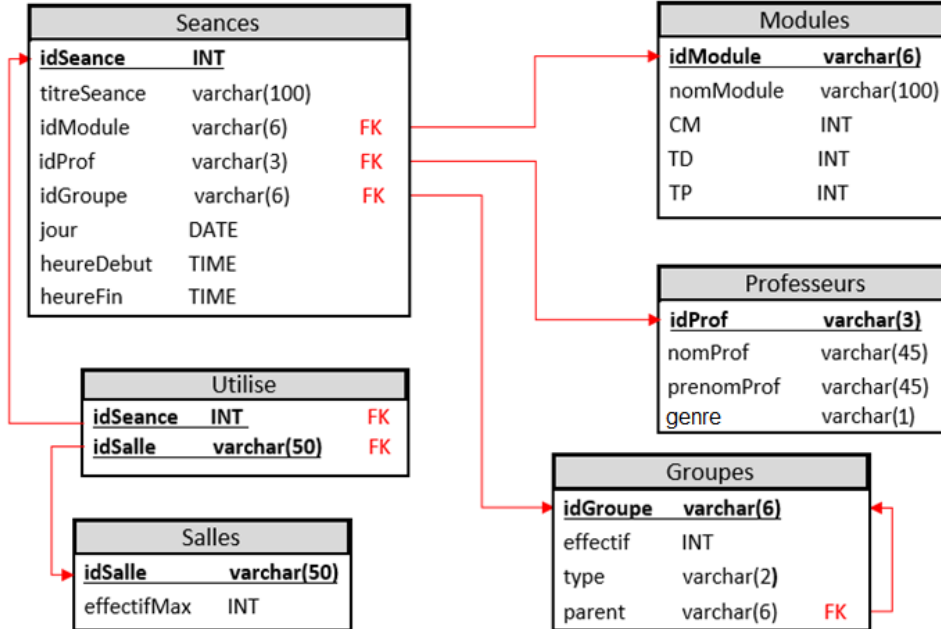
Aperçu :

nom	nom des bibliothèques
PERRIN	CAMUS,VAILLAND,CESAIRE
PETIT	ZOLA,PROUST

```
SELECT e.nom, group_concat(b.nom) "nom des bibliothèques"
FROM Emprunteurs e JOIN Villes ON ville=idville
      JOIN Bibliotheques b ON b.ville=Villes.idville
GROUP BY e.nom;
```

EXERCICE 2 (sur la base EDT)

On rappelle le schéma de la base EDT :



1) Créer une requête affichant pour chaque séance, le nombre d'étudiants concernés

ATTENTION : Vous donnerez si possible deux solutions :

- l'une avec NATURAL JOIN
- l'autre avec JOIN...ON...

Aperçu du résultat attendu :

idSeance	titreSeance	idModule	idProf	idGroupe	effectif	jour	heureDebut	heureFin
15719	SS1	M1106	BTE	S1	76	2021-01-12	13:45:00	17:45:00
16350	Algo prog	M1103	EDU	S1	76	2020-11-16	11:00:00	12:30:00
20644	SS1	M1106	NULL	S1	76	2021-01-11	13:45:00	17:45:00
20646	SS1	M1106	NULL	S1	76	2021-01-13	08:30:00	12:30:00

```

select idSeance,titreSeance,idModule,idProf,Seances.idGroupe,effectif,jour,heureDebut,heureFin
from Seances JOIN Groupes
ON Seances.idGroupe=Groupes.idGroupe;
  
```

```

select idSeance,titreSeance,idModule,idProf,Seances.idGroupe,effectif,jour,heureDebut,heureFin
from Seances NATURAL JOIN Groupes;
  
```

2) On s'intéresse à cette requête n°1 :

```

SELECT *
FROM (Seances NATURAL JOIN Groupes) NATURAL JOIN Professeurs
  
```

Cette requête n°1 peut-elle s'écrire sous la forme ci-dessous ? Si oui, complétez les pointillés. Si non expliquez.

```

SELECT *
FROM Seances JOIN Groupes ON ...
JOIN Professeurs ON ...
  
```

SELECT *

FROM Seances JOIN Groupes ON Seances.idGroupe=Groupes.idGroupe
JOIN Professeurs ON Seances.idProf=Professeurs.idProf

3) On s'intéresse à cette requête n°2 :

SELECT *

FROM (Groupes NATURAL JOIN Professeurs) NATURAL JOIN Seances

a) La requête n°2 peut-elle s'écrire sous la forme ci-dessous ? Si oui, complétez les pointillés. Si non expliquez.

SELECT *

FROM Groupes JOIN Professeurs ON ...
JOIN Seances ON ...

Pas possible car on ne peut faire de jointure entre Groupes et Professeurs !

b) La requête n°2 peut-elle s'écrire sous la forme ci-dessous ? Si oui, complétez les pointillés. Si non expliquez.

SELECT *

FROM (Groupes NATURAL JOIN Professeurs)
JOIN Seances ON ...

SELECT *

FROM (Groupes NATURAL JOIN Professeurs)
JOIN Seances ON Seances.idGroupe=Groupes.idGroupe
AND Seances.idProf=Professeurs.idProf

Par contre, comme Groupes et Professeurs n'ont aucun attribut de même nom, Groupes NATURAL JOIN Professeurs retourne le produit cartésien. En faisant ensuite NATURAL JOIN Seances, la jointure se fera sur 2 attributs.

c) Les requêtes n°1 et n°2 renvoient-elles le même résultat ?

Oui

4) Créer une requête affichant pour chaque séance la liste des salles où elle se déroule.
Vous donnerez si possible deux solutions l'une avec NATURAL JOIN l'autre avec JOIN...ON...

Aperçu du résultat attendu :

idSeance	titreSeance	idmodule	idprof	idgroupe	jour	heureDebut	heureFin	salles
17	COM	M3205	ADU	S3G2	2020-10-21	10:30:00	12:30:00	IN0.09
721	COM	M3205	ADU	S3G3	2020-10-14	13:45:00	15:45:00	IN0.07
782	Init Algo	M1102	HULL	S1G2	2020-09-14	08:30:00	10:30:00	IN0.06,IN0.07

select Seances.idSeance,titreSeance,idmodule,idprof,idgroupe,jour,heureDebut,heureFin,group_concat(idSalle) salles
from Seances join Utilise
on Seances.idseance=Utilise.idSeance
group by idSeance;

select Seances.idSeance,titreSeance,idmodule,idprof,idgroupe,jour,heureDebut,heureFin,group_concat(idSalle) salles
from Seances NATURAL JOIN Utilise
group by idSeance;

EXERCICE 3 (sur la base **EDT**)

1) Créer une requête affichant la liste de toutes les séances pour lesquelles le nombre d'étudiants présents dépasse la capacité des salles réservées.

Attention : vous tiendrez compte du fait qu'une séance peut utiliser plusieurs salles

23 lignes dans le résultat.

Aperçu du résultat attendu :

idSeance
22405
35159
35161

```
select se.idSeance,sum(effectifmax),effectif
from Seances se join Utilise ut on se.idseance=ut.idSeance
              join Salles sa on ut.idsalle = sa.idsalle
              join Groupes gr on se.idgroupe=gr.idgroupe
group by se.idSeance
having effectif > sum(effectifmax);
```

2) Créer une requête affichant les groupes qui sont inclus dans les autres. (On supposera qu'une inclusion est au maximum de niveau 2)

Aperçu du résultat attendu :

ce groupe	est inclus dans le groupe
S1G1	S1
S1G2	S1
S1G3	S1
S1G1.1	S1
S1G1.1	S1G1

```
select g1.idgroupe 'ce groupe',g3.idgroupe 'est inclus dans le groupe'
from Groupes g1 join Groupes g2
              on g1.parent=g2.idgroupe
              join Groupes g3
              on (g1.parent=g3.idgroupe or g2.parent=g3.idgroupe);
```

3) On dira que deux séances se **chevauchent** si leurs plages horaires se coupent, càd partagent au moins 1 seconde.

Exemples :

- les plages horaires ["10:45:00" , "12:30:00"] et ["08:30:00" , "09:30:00"] ne se coupent pas.
- les plages horaires ["10:45:00" , "12:30:00"] et ["08:30:00" , "10:45:00"] ne se coupent pas.
- les plages horaires ["10:45:00" , "12:30:00"] et ["08:30:00" , "11:00:00"] se coupent.
- les plages horaires ["10:45:00" , "12:30:00"] et ["08:30:00" , "10:45:01"] se coupent.

On dira que deux séances sont **en conflit** si elles se chevauchent et si, en plus de cela, elles utilisent une même salle ou un même prof.

Créer une requête retournant les paires de séances qui sont **en conflit**.

Aperçu du résultat attendu :

idseance	idseance	conflit de salles	conflit de prof
8752	51658	IN1.05	NULL
22405	69085	GT1.07a,GT1.07b	NULL
35537	91344	Amphithéâtre Le Bugey	NULL
35877	65264	NULL	ADA
88637	90048	IN1.09	NULL
88637	90051	IN1.07	NULL

!/\\ vous n'avez pas à savoir refaire cette question qui est difficile !/

```

SELECT distinct
  s1.idseance,
  s2.idseance,
  group_concat(distinct CASE WHEN u1.idsalle=u2.idsalle THEN u1.idsalle END) 'conflit de salles',
  group_concat(distinct CASE WHEN s1.idprof=s2.idprof THEN s1.idprof END) 'conflit de prof'
FROM Seances s1 NATURAL JOIN Utilise u1
  JOIN (Seances s2 natural join Utilise u2)
  ON s1.jour=s2.jour
  AND (u1.idsalle=u2.idsalle OR s1.idprof=s2.idprof)
  AND (s1.heuredebut<s2.heureFin)
  AND (s2.heuredebut<s1.heurefin)
WHERE s1.idseance<s2.idseance
GROUP BY s1.idseance,s2.idseance;

```